

Two findings on the Micromart book

The dormancy service moved 175 customers it should not have — Micromart is right, and the cause is not the one they named. Separately, the arrears report’s penalty column is not “almost zero”; it is structurally incapable of being anything else.

11 September 2026 · applied and verified Serviceconnect · 100.72.35.56,4230

All figures read live, read-only

ISSUE 1 · DORMANCY

Micromart’s complaint is correct

Of the 500 customers moved at 02:34 this morning, 175 had settled their loans within the last 60 days. One cleared a four-year-old debt yesterday afternoon and was moved out overnight. **Rolled back and the clock fixed the same day** — bar the SMS and the PIN, which could not be recalled.

ISSUE 2 · PENALTIES

The CEO is right, for four reasons at once

The arrears penalty column reads **0.00 on every one of 16,542 rows**, and always will. The report’s date pickers do nothing. And 99.9% of the book is on a product that can never be penalised at all.

Issue 1 · The dormancy clock

What the service did last night

The job fired at 02:34:54 and moved **500 customers** and 2,188 loans from Micromart Africa (3002) to Micromart Fintech (3005). Five hundred is not a coincidence — it is the `@MaxBorrowers` ceiling, so the run was truncated and a backlog remained.

500

customers moved, the ceiling exactly

175

moved wrongly — settled under 60 days ago

325

genuinely dormant; these stay put

1

shortest true dormancy, in days

Micromart’s reading of the rule is the right one

Script 10 started the 60-day clock at `Loans.ExpectedClearDate` — the date a loan was *scheduled* to mature. Micromart say the clock starts when the customer actually finished paying. For a customer who pays on time those two dates are days apart and the difference is invisible. For an NPL customer who runs two years late and then settles, they are years apart, and the service reads the delinquency itself as evidence of dormancy. That is exactly backwards: the longer someone struggled before paying, the faster the job threw them off their officer’s book.

The two customers Micromart sent over demonstrate it cleanly.

BORROWER	PHONE	LAST LOAN	MATURED	JOB SAID	ACTUALLY SETTLED	TRULY DORMANT	ELIGIBLE FROM
132659	254742462695	392908	2026-01-28	226 d	2026-09-02	9 d	2026-11-01
148839	254714230384	420215	2026-06-18	85 d	2026-09-03	8 d	2026-11-02

Borrower 132659’s ledger is worth reading in full, because it shows the shape of the problem. The loan matured in January. They kept paying in 200- and 300-shilling instalments through the year, absorbed a 340-shilling late penalty in March, and brought the balance to zero on 2 September — nine days ago. The service called them 226 days dormant.

```
-- dbo.CustomerStatement, loan 392908, tail of the ledger
2026-03-10  Loan Repayment      200  balance 1700
2026-03-31  Late Payment Penalty 340  balance 2040  ← 62 days past maturity
2026-04-07  Loan Repayment      400  balance 1640
...
2026-08-30  Loan Repayment      200  balance  140
2026-09-02  Loan Repayment      140  balance   0   ← the real clear date
```

The worst individual case in the run is not one of those two. Borrower 5485 had a loan that matured on 2022-07-25. The job called them 1,509 days dormant and moved them. They had cleared that loan on 2026-09-10 — the previous afternoon.

How the 175 break down

HOW LONG AGO THEY ACTUALLY SETTLED	CUSTOMERS	ASSESSMENT
Cleared within the last 7 days	26	Fresh recoveries, moved the same week they paid
8–14 days	16	Still actively repaying customers
15–30 days	42	Well inside the 60-day window
31–59 days	91	Inside the window, closest to the line
Wrongly moved	175	Roll back
60+ days since settlement	325	Correct under both the old and new rule — leave them

Why only 175 go back, not all 500

The other 325 qualify under the corrected rule too, and they have already been notified. Rolling the whole run back would re-move them tonight and send every one of them a second, identical SMS. Reversing only the customers the rule actually protects is both more accurate and quieter.

The fix cannot use the column you would expect

`Loans.DateCleared` exists and is exactly the right name for what we need. It is also NULL on all 344,332 loans in the book — 205,162 of them flagged cleared, none with a date. ServiceSuite has never written to it. Anything built on that column would have silently moved nobody, or everybody.

So the ledger is the only witness. `CustomerStatement` carries a running `LoanBalance` per transaction, so the moment a loan hit zero is the `TransactedDate` of the row where that balance reads 0. The

corrected procedure takes, per loan, the *last* such moment — not the first, because a late-payment penalty can push a settled loan back above zero and the customer then pays it down again — and per customer, the latest across all their loans.

Three cases have to be told apart, and the distinction matters:

CASE	CUSTOMERS	TREATMENT
Ledger shows a zero row	883	Use that date. This is the normal path.
No ledger rows at all — pre-2022 loans that predate <code>CustomerStatement</code>	8	Fall back to maturity. Nothing can contradict it, and these really are long dormant.
Ledger exists but never reaches zero	2	Hold. We cannot date the settlement, and moving someone we cannot date is how this went wrong. Holding is reversible; moving is not.

What can and cannot be undone

✓	The customer record — fully reversible <code>EntityId</code> , <code>EntityUnit</code> and <code>EntityAgent</code> restore from <code>MicroEazyDormancyLog</code> . Verified: <code>FromUnit</code> and <code>FromAgent</code> are populated on all 500 rows, and <code>Borrowers.OldAgent</code> independently agrees with the log on all 500. Every customer goes back to the officer who actually held them.
✓	The ledger — fully reversible 889 loans and their children: 4,174 <code>loanSchedule</code> rows, 83 <code>loanChargesSchedule</code> , 10,835 <code>CustomerStatement</code> , 18,851 <code>Journals</code> , plus <code>ManagedLoans</code> , <code>LoanGraduationHistory</code> and <code>STKPaymentRequests</code> .
✓	The e-mails — caught in time 205 notification e-mails were queued and none have been sent . 83 of them belong to the wrongly-moved customers and are pulled from the queue by the rollback.
✗	The SMS — already gone All 500 messages were drained and marked sent within the hour. 175 customers have been told their account moved to Micromart Fintech when it has not . Whether they get a correction message is Micromart's call, not ours — but they need to be asked today, because the customers already have the first message.
✗	The USSD PIN — not recoverable The move set <code>UssdPin</code> and <code>MobileAppPassword</code> to NULL; 481 of the 500 are still NULL. The old value was a BCrypt hash and there is nothing to restore it from. In practice this is mild — a NULL PIN is a supported state and the USSD service walks the customer through setting a new one on their next dial — but it is not a reversal, and Micromart should hear it from us rather than discover it.

How it was done

Both scripts were applied on 11 Sep and verified afterwards

The analysis above came entirely from read-only queries; the founder then ran the two scripts — rollback committed 15:45:20, procedure replaced 15:49:12. The steps below are the sequence that was followed, and the verification that closes this section is the proof it landed.

1 Dry-run the rollback

`fintech-migration\12-rollback-wrong-dormancy.sql` defaults to `@DryRun = 1`. It prints the 175 with the date the job used, the date they actually settled, and the date each becomes genuinely eligible. Send that list to Micromart and let them confirm it against their own view before anything moves.

2 Apply it in a quiet window

Flip `@DryRun` to 0. `CustomerStatement` (3.5M rows), `Journals` (6.8M), `loanSchedule` (2.0M) and `loanChargesSchedule` (618k) are heaps with *no index at all*, so every update is a full scan — roughly 13M rows inside one transaction. Do not run it mid-morning on a shared box. The script ends with four checks that must all read zero.

3 Install the corrected procedure

`13-fix-dormancy-clock.sql` replaces `sp_MicroEazy_MigrateDormantToFintech` in place. The 02:30 job created by script 10 calls it by name and needs no change, so installing the procedure is all tonight's run needs.

4 Dry-run the new rule before 02:30

`EXEC dbo.sp_MicroEazy_MigrateDormantToFintech;` — the default is a dry run. Simulated against the post-rollback book it selects 55 **customers**, holds 2, and re-selects **none** of the 175. Well under the 500 ceiling, so no truncation.

5 Ask Micromart about the SMS

175 people have a message saying something that is no longer true. That is a decision for them, and it is the only part of this that cannot wait.

If the procedure is not replaced before tonight

The old rule, run against the book as it stands after a rollback, would move 265 **customers** at 02:30 — including all 175 again, plus roughly 35 more of the same kind. The rollback alone does not stop the bleeding; the procedure has to go in with it.

Verified after the run

Both scripts were applied: the rollback committed at 15:45:20 and the procedure was replaced at 15:49:12. Re-checked live against the book:

CHECK	EXPECTED	ACTUAL
Customers marked reversed	175	175 pass
Reversed customers now on 3002	175	175 pass
Correctly-moved customers left on 3005	325	325 pass
Restored to their original officer / branch	175	175 / 175 pass
Their loans, schedules, statements left on 3005	0	0 pass
Unsent move e-mails belonging to them	0	0 pass
Move e-mails actually <i>sent</i> to a reversed customer	0	0 pass
Procedure now using the settlement clock	yes	yes pass

A dry run of the installed procedure selects **55 customers** in 2.1 seconds, holds 2 as undatable, and reports a minimum true dormancy of **exactly 60 days** – none below the threshold, and comfortably under the 500 ceiling. The fix also corrects in the other direction: the shortest *old-rule* figure in that set is 45 days, so several customers who repaid early and have genuinely been idle since were being held back by the maturity clock and now move correctly.

Two customers borrowed on the fintech book before the rollback caught them

Both were in the wrongly-moved 175, both got the 02:34 SMS telling them they could now borrow from home, and both did exactly that during the day – before the 15:45 rollback pulled them back to 3002 along with their new loan. Each now holds a **Micro Eazy product on the Micromart Africa book**:

- **132369** – loan 450567, *Micro Chap Chap*, **disbursed at 13:02**, KES 7,000 outstanding, due 2026-10-09. Real money, live exposure, wrong book.
- **116744** – loan 450461, *Micro Eazy*, KES 9,125, **applied for but not disbursed** (approval stage 2058). No money has moved, but the application will now be worked by 3002's branch rather than the fintech desk.

Nothing is broken today, but the product belongs to entity 3005 and the loan sits on 3002, so neither loan will report correctly and the product's rollover configuration will not match the book it is on. **Recommendation: move these two back to 3005** using `sp_MicroEazy_MoveCustomerToFintech` from script 11, which does the same move for one named customer without the 60-day clock. They have demonstrated by their own action that they are fintech customers. Tonight's job will not pick either of them up – both now have an open loan, so the corrected rule excludes them.

One thing that could not be verified from here, and one pre-existing defect

The SQL Agent job itself could not be confirmed: the `service_suite_user` login has no SELECT rights anywhere in `msdb`, so `sysjobs`, `sysjobsteps` and `sysjobhistory` are all closed to us. **Someone with admin rights should eyeball the job in SSMS before tonight** – that it is still enabled, still scheduled for

02:30, and still points at `sp_MicroEazy_MigrateDormantToFintech`. The procedure it calls is confirmed correct; only the trigger is unverified.

Separately: 246,146 `Journals` rows across the whole book carry a `ProductId` in the `EntityId` column (30 distinct product ids, from Sep 2023 to a row written at 15:50 today). That is ServiceSuite's own fee-posting path, not anything our scripts did — but it means fee journals are invisible to every entity-based restamp, in both directions. Worth raising with Micromart separately; it is not ours to fix.

Issue 2 · Why the arrears report shows no penalties

The CEO’s three statements are each partly right, and the truth underneath them is worse than any of them. Taking his points in order.

“The penalty column will have almost 0”

Not almost. **Exactly zero, on every row, permanently.** The arrears and NPL reports partition the book on the same expression, and they do not overlap:

```
-- sp_arrearsLoans
WHERE DATEDIFF(DAY, L.ExpectedClearDate, GETDATE()) < 90

-- sp_NplReport
WHERE DATEDIFF(DAY, ExpectedClearDate, getdate()) > 90
```

And every penalised loan in the book sits on the far side of that line. This is the whole of `Loans`, bucketed:

DAYS PAST MATURITY	OPEN LOANS	WITH A PENALTY	CLEARED LOANS	WITH A PENALTY
Not yet matured	10,410	0	688	0
0–59 past due	3,408	0	10,403	0
60–89 past due	1,588	0	5,358	0
90+ past due (NPL)	60,453	29,424	250,670	740

All 30,164 loans returned by `select * from loans where Penalty>0` are in the bottom row. There is not one penalised loan anywhere in the 0–89 day range that the arrears report covers. Running both procedures against entity 3002 confirms it end to end:

16,542

rows on the arrears report

0.00

penalty total on that report

49,511

rows on the NPL report

24.73M

KES of penalty, all of it there

“We penalize loans past 60 days, so all loans past 60 days should have a figure”

The intent is configured correctly — and then defeated four separate times. Each of these alone would be enough to empty the column.

1

Almost no product is eligible — the big one

The engine is `sp_GeneralRolloverService`, and its first condition is `P.RollOverApplication = 1`. That column is the “Penalty application option” radio on the product screen (1 = on loan maturity, 2 = on instalments). It is **NULL on 28 of Micromart Africa’s 34 products and on all three Micro Eazy products** — the products were saved with “apply penalty = yes” ticked but the radio never chosen. `NULL = 1` is never true, so those products are invisible to both penalty services. Of the 1,246 open loans currently sitting 60–89 days past due, **exactly one** is on a product that could ever be penalised.

A loan can only be penalised once, ever

2	The same procedure filters on <code>L.IsRolledOver = 0</code> and then sets <code>IsRolledOver = 1</code> . There is no reset. A loan is penalised once at day 61 and never again, however long it sits. Run the engine right now and it selects zero loans — everything eligible has already had its single penalty.
3	The service has not really run since June Bulk penalty postings stop on 2026-06-24 (2,760 rows that day); after that only a trickle of one to twenty-two a day, which is manual rollovers, not the service. Any loan that crossed day 61 since late June has never been visited. That is precisely the population the arrears report covers.
4	A third of the penalties that were charged are not in the column Both rollover procedures write <code>Penalty = Penalty + T.PenaltyAmount</code> . Where <code>Penalty</code> starts NULL, <code>NULL + 340</code> is NULL — the balance goes up, the ledger row is written, the customer is charged and texted, and the column stays empty. 45,416 loans carry a penalty in the ledger; 15,252 of them (33.6%) show NULL in <code>Loans.Penalty</code> . Loan 392908 above is one of them: charged 340 in March, column still NULL. Note the column is never <code>0</code> — always NULL or a real figure, which is the signature of exactly this bug.

“Does the start date and end date make a difference?”

No. In both procedures `@startdate` and `@enddate` are declared in the signature and then **never referenced again anywhere in the body** — not in a filter, not in a computed column, not in the branch selection. Whatever the CEO picks, he gets the same rows, evaluated against `GETDATE()` at the moment he runs it.

This matters more than it looks. It means the arrears and NPL reports are not *as-at* reports at all: they cannot be run for last month, they cannot be reconciled against a prior close, and two people running them on different days get different numbers with no way to say why. The date pickers are, in effect, decoration.

“Should a loan in NPL or arrears with a penalty appear in both reports?”

It cannot, and it should not. The two reports are a partition on days-past-maturity: under 90 is arrears, over 90 is NPL. A loan is in exactly one. If a penalty ever does show up in the arrears report, it will be a loan penalised at day 61 that has not yet aged past day 90 — a 29-day window, and today nothing is in it.

Two smaller defects found while confirming this, both worth fixing while the file is open:

DEFECT	SCALE	EFFECT
The boundary is <code>< 90</code> and <code>> 90</code> , so loans at <i>exactly</i> 90 days fall through the gap	16 loans	Invisible on both reports. One should use <code>>=</code> .
The arrears report has no <code>LoanCleared = 0</code> filter — the NPL report does	10,235 of 16,542 rows	62% of the arrears report is already-settled loans. The CEO is chasing a list that is mostly closed business.

What to do about it

Do not simply switch the products on

Setting `RollOverApplication = 1` on the products that already have `RollOverPenalty = 1` would make the engine select 2,794 loans on its next run, against KES 16.46M of balance, and charge **KES 3.29M in penalties** — every one of them adding to a live customer’s balance and sending that customer an SMS saying

so. Much of it would land on loans that went past due months or years ago. That is a commercial and conduct decision for Micromart's board, not a configuration fix, and it should never be made by ticking a radio button on a product screen.

The sequence that gets the CEO a report he can trust, cheapest first:

1 Repair the reports — no customer impact, do this first

Add `LoanCleared = 0` to `sp_arrearsLoans`, close the 90-day boundary gap, and either wire `@startdate / @enddate` into the filter or remove them from the screen. These are ServiceSuite's own procedures, so the change belongs to them; the finding is ours to hand over. On its own this cuts the arrears report from 16,542 rows to 6,307 real ones.

2 Backfill the 15,252 lost penalty figures

The money was charged and the ledger has every row, so `Loans.Penalty` can be rebuilt from `CustomerStatement` exactly. No customer is affected — it is a reporting column catching up with what already happened. Fix `Penalty = ISNULL(Penalty,0) + ...` in both rollover procedures at the same time, or it will drift straight back out.

3 Then decide the policy question, with the numbers in front of them

Whether Micromart wants a once-only penalty at day 61 or a recurring one; whether it applies to all products or a few; and whether the 2,794 loans now sitting past grace get charged retrospectively or only new arrears going forward. Until that is answered, the penalty column will keep telling a story about configuration rather than about credit.

The one-line answer, if it needs to be said in a meeting

The arrears report shows no penalties because penalties are only ever applied after day 60 and the report stops at day 90 — but the deeper reason is that the penalty engine has been effectively switched off since June and can only see six of Micromart's thirty-seven products. The 30,000 penalised loans the CEO can see in the table are historical, and every one of them is already in NPL.